

GitHub Issues Connector – Local Debugging (Full Narrative with Screenshot Placeholders)

Summary of Key Points

This document is a comprehensive technical record of several days of local debugging for the GitHub Issues Connector, a Microsoft Graph External Connector built on Azure Functions. All work was done locally using Visual Studio Code, without an Azure subscription or Microsoft 365 tenant.

The core challenges were not related to TypeScript syntax, ports, or missing files, but to Azure Functions startup lifecycle behaviour: eager Azure Identity creation, module-load exceptions, and Timer triggers requiring Azure Storage. The final resolution mirrors the PnP OAuth sample model by explicitly disabling Azure-dependent components during local development while preserving production wiring.

Background

The debugging effort followed the Microsoft Learn module 'Build your first Microsoft 365 Copilot Connector', but with intentional deviations from the listed prerequisites. Specifically, the work proceeded without Azure credentials, a Copilot-enabled tenant, Teams publish capability, or Azure Storage. The goal was to understand how far the project could run locally and to align it architecturally with the PnP samples.

Phase 1 – TenantId and Azure Identity Startup Failure

The initial failure manifested as:

'ClientSecretCredential: tenantId is a required parameter'

This appeared alongside misleading messages such as 'Unable to load entry point' and 'No job functions found'. A key realisation was that Azure Functions reports these errors when a top-level exception occurs during module evaluation, not when a file is missing.

Phase 2 – Why config.ts Changes Were Insufficient

Early fixes focused on config.ts by relaxing validation and skipping identity checks in Development mode. While these changes were correct, they did not prevent the crash. The decisive insight was that Azure Identity was instantiated earlier, inside graphClient.ts, before any Development guards could apply.

Phase 3 – graphClient.ts as the Decisive Fix

The final resolution in this phase was to adopt the PnP-style pattern of never instantiating Azure credentials during local development. Instead of throwing errors, graphClient.ts now returns null when in Development, allowing the Functions host to complete startup and only failing if Graph functionality is actually invoked.

JoeDebug Terminology

JoeDebug2: Launch Azure Functions using func.exe start (portable). This mode exposes startup failures and is the primary debugging strategy.

JoeDebug3: Attach-only Node debugger on port 9229. This requires a pre-running host and was intentionally not used in this exercise.

Phase 4 – Timer Triggers and Azure Storage

After identity issues were resolved, Azure Functions proceeded further and encountered a new failure:

'No connection could be made because the target machine actively refused it (127.0.0.1:10000)'

This error indicates that Timer triggers require Azure Storage. Because AzureWebJobsStorage was set to UseDevelopmentStorage=true and Azurite was not running, the Functions host could not complete startup and therefore never bound to HTTP port 7071.

/api Routing and Port Clarification

The /api route is virtual and injected by the Azure Functions host; it does not correspond to any folder on disk. Port 7071 is the default HTTP port. A browser message stating 'localhost refused to connect' means the host never finished starting, not that routing or ports were misconfigured.

Screenshot Placeholders (Insert Screenshots Below)

Screenshot 1: VS Code showing dist/src/functions/pingJoe.js – confirms successful TypeScript compilation.

Screenshot 2: Browser displaying 'localhost refused to connect' at http://localhost:7071/api/pingJoe – shows host never bound to HTTP due to Timer failure.

Screenshot 3: Terminal output showing Timer listener failure at 127.0.0.1:10000 – evidences missing Azurite/storage.

Screenshot 4: graphClient.ts with Joe-style Development guard returning null – decisive fix preventing tenantId crashes.

Final Recommended Local Development Posture

For local development without Azure:

- Disable Timer triggers in Development
- Keep HTTP-only endpoints enabled (e.g., pingJoe)
- Skip Azure Identity and Graph calls locally
- Re-enable full Azure wiring only when a tenant and storage are available

References

- Microsoft Learn: Build your first Microsoft 365 Copilot Connector
- Azure Functions Node.js runtime lifecycle documentation
- PnP Copilot pro-dev samples (OAuth-off Development pattern)